

◆ CipherForge

تطبيق تشفير شامل بأربع خوارزميات — موقع ويب وبرنامج سطح مكتب

دليل المشروع وشرح الأكواد + أسئلة المناقشة

الطالبة:

المقرّر:

الأستاذ:

التاريخ:

1) نظرة عامة على المشروع

CipherForge تطبيق يقوم بتشفير وفك تشفير أي بيانات (نصوص، ملفات، صور، صوت حتى 50 ميغابايت) باستخدام أربع خوارزميات تشفير مختلفة، ويعمل بطريقتين: كموقع ويب عبر المتصفح، وكمبرنامج سطح مكتب (EXE) عبر Electron.

الهدف الأكاديمي: المقارنة العملية بين عائلات التشفير المختلفة وتطبيق مبادئ الأمان الحديثة.

الخوارزمية	النوع	الاستخدام
AES-256-GCM	متماثل + توثيق (AEAD)	كلمة مرور
ChaCha20-Poly1305	متماثل حديث + توثيق (AEAD)	كلمة مرور
Triple-DES-CBC	متماثل كلاسيكي + HMAC	كلمة مرور
RSA-OAEP-2048	غير متماثل (هجين)	زوج مفاتيح

2) التقنيات المستخدمة

الجزء	التقنية	الدور
الخادم	Node.js + Express 5	استقبال الطلبات وتنفيذ التشفير
التشفير	مكتبة crypto المدمجة	تنفيذ الخوارزميات
اشتقاق المفاتيح	scrypt (N=16384)	تحويل كلمة المرور لمفتاح قوي
رفع الملفات	multer 2 (تخزين بالذاكرة)	استقبال الملفات

الأمان	helmet + express-rate-limit	حماية الرؤوس وتحديد الطلبات
الواجهة	HTML/CSS/JS صرّف	واجهة المستخدم
سطح المكتب	electron + electron-builder	إنتاج برنامج EXE

(3) هيكل المشروع

```

server.js      // الخادم: يربط الأمان + الواجهة + المسارات
main.js        // نافذة سطح المكتب (Electron تغليف)
public/        // الواجهة (index.html, css, js)
src/
  services/     // أكواد التشفير الأربعة + اشتقاق المفاتيح
    aesGcm.js  chacha20.js  tripleDes.js  rsaOaep.js  keyDerivation.js
  routes/       // نقاط الـ API
    encryption.js  keys.js  algorithms.js
  middleware/    // رفع الملفات + معالجة الأخطاء
  utils/         // (envelope) الثوابت + المغلّف

```

4) المفاهيم الأمنية الأساسية

المصطلح	الشرح
متماثل (Symmetric)	نفس المفتاح/كلمة المرور للتشفير والفك. سريع. (AES, ChaCha, 3DES)
غير متماثل (Asymmetric)	مفتاحان: عام للتشفير، خاص للفك. (RSA)
Salt (الملح)	قيمة عشوائية تُخلط مع كلمة المرور كي لا تنتج نفس النتيجة مرتين أبدًا.
IV / Nonce	قيمة عشوائية لكل عملية تشفير تمنع تكرار النمط.
KDF	دالة تحوّل كلمة المرور (الضعيفة) إلى مفتاح تشفير قوي. هنا: <code>scrypt</code> .
AEAD	تشفير + توثيق معًا: يضمن السريّة وأن البيانات لم تُعدّل.
Auth Tag	بصمة توثيق؛ لو تغيّر أي بايت أو كانت كلمة المرور خاطئة يفشل الفك.
MAC / HMAC	توقيع للتأكد من سلامة البيانات (نستخدمه يدويًا مع 3DES).

5) شرح الخوارزميات بالتفصيل

أ) اشتقاق المفتاح — `keyDerivation.js`

لا نستخدم كلمة المرور كمفتاح مباشرة، بل نمرّرها عبر `scrypt`:

```
crypto.scryptSync(password, salt, keylen, { N: 16384, r: 8, p: 1 });
```

- يجعل العملية بطيئة عمدًا → يُصعب تجربة ملايين كلمات المرور (مقاومة Brute-force). `N=16384`
- المعامل `info` يتيح اشتقاق **مفاتيح منفصلين** من نفس الملح (مفتاح تشفير + مفتاح توقيع) عبر دمجهم في الملح.

ب) AES-256-GCM — `aesGcm.js`

```
const salt = randomBytes(16); // ملح عشوائي
const iv = randomBytes(12); // عشوائي IV
const key = deriveKey(password, salt, 32);
const cipher = createCipheriv('aes-256-gcm', key, iv);
cipher.setAAD(AAD);
const data = update + final; // التشفير
const tag = cipher.getAuthTag(); // بصمة الحماية
```

عند الفك نضع `setAuthTag(tag)`؛ فإن كانت كلمة المرور خاطئة أو البيانات مُعدّلة → يفشل الفك تلقائيًا. هذا معنى GCM (تشفير + توثيق).

ج) ChaCha20-Poly1305 — `chacha20.js`

نفس فكرة AES لكنها خوارزمية حديثة أسرع على الأجهزة بلا تسريع AES عتادي (كالهواتف). الفرق التقني:

```
cipher.setAAD(AAD); // بدون خيار plaintextLength
```

د) Triple-DES-CBC — tripleDes.js (Encrypt-then-MAC)

كلاسيكية وبلا توثيق مدمج، فأضفنا الحماية يدويًا:

```
const ciphertext = cipher(...); // 1) شَفِّر
const mac = HMAC-SHA256(macKey, iv + ciphertext); // 2) وُقِّع
```

وعند الفك نتحقّق من التوقيع **أولًا** بمقارنة زمنية ثابتة ثم ن فكّ:

```
if (!crypto.timingSafeEqual(mac, expected)) throw 'بيانات تالفة';
```

مفتاح التشفير ومفتاح التوقيع **منفصلان** — مبدأ أمني مهم (Key Separation).

هـ) RSA-OAEP-2048 — rsaOaep.js (تشفير هجين)

RSA بطيئة ولا تشفّر إلا بيانات صغيرة، فنستخدم الأسلوب الهجين:

1. ننشئ مفتاح AES عشوائيًا.
 2. نشفّر البيانات الفعلية بـ AES (سريع).
 3. نشفّر مفتاح AES الصغير فقط بـ RSA باستخدام المفتاح العام.
- وعند الفك: المفتاح الخاص يفكّ مفتاح AES، ثم AES يفكّ البيانات.

6) رحلة الطلب (كيف يعمل التطبيق)

1. المستخدم يختار الخوارزمية ويدخل النص/الملف وكلمة المرور في الواجهة.
2. الواجهة (`js/app.js`) ترسل الطلب إلى الخادم عبر `js/api.js`.
3. المسار `routes/encryption.js` يستقبل الطلب ويختار الخدمة المناسبة.
4. خدمة التشفير تنقذ العملية وترجع **مغلّفًا** (envelope) فيه: الملح + IV + الوسم + البيانات.
5. المغلّف يُحوّل إلى نص base64 واحد يسهل نسخه أو حفظه.

7) أسئلة المناقشة المتوقعة + الأجوبة

? لماذا اخترت هذه الخوارزميات الأربع تحديداً؟

✦ لتغطية كل العائلات: **AES** (المعيار العالمي المتمثل)، **ChaCha20** (بديل حديث سريع)، **3DES** (كلاسيكي للمقارنة التاريخية)، و **RSA** (غير متمثل). هذا يظهر فهماً شاملاً لأنواع التشفير.

? ما الفرق بين التشفير المتمثل وغير المتمثل؟

✦ المتمثل يستخدم **مفتاحاً واحداً** للتشفير والفك (سريع، لكن يجب مشاركة المفتاح بأمان). غير المتمثل يستخدم **مفتاحين**: عام للتشفير وخاص للفك (يحل مشكلة تبادل المفاتيح لكنه أبطأ).

? ما معنى GCM؟ ولماذا اخترتها بدل CBC في AES؟

✦ GCM وضع تشغيل يوفر **التشفير والتوثيق معاً (AEAD)**؛ أي يضمن السرية **وكشف أي تعديل** على البيانات. CBC يشفر فقط دون توثيق، فيكون عرضة لهجمات التلاعب. لذا GCM أكثر أماناً.

? ما هو الـ IV ولماذا يجب أن يكون عشوائياً؟ وما الفرق بينه وبين الملح؟

✦ الـ IV قيمة تبدأ بها عملية التشفير؛ عشوائيتها تضمن أن تشفير نفس النص مرتين يعطي نتيجتين مختلفتين (يمنع تسريب الأنماط). الفرق: **الملاح** يُستخدم في اشتقاق المفتاح من كلمة المرور، والـ **IV** يُستخدم في عملية التشفير نفسها. كلاهما عشوائي ويُخزن مع الناتج.

? لماذا لا تستخدم كلمة المرور كمفتاح مباشرة؟ وما هو scrypt؟

✦ لأن كلمات المرور قصيرة وضعيفة وغير موزعة عشوائياً. **scrypt** دالة اشتقاق مفاتيح تحوّلها إلى مفتاح قوي بطول ثابت، وهي **بطيئة ومكلفة للذاكرة عمداً** (المعامل $N=16384$) لثُغيب هجمات التخمين الجماعي.

? ماذا يحدث لو أدخل المستخدم كلمة مرور خاطئة عند الفك؟

✦ يفشل الفك ويظهر خطأ واضح. في AES/ChaCha يفشل التحقق من **Auth Tag**، وفي 3DES يفشل التحقق من **HMAC**. لا تظهر بيانات مشوّهة بل رفض صريح.

? لماذا استخدمت RSA بشكل هجين ولم تشفّر كل البيانات بـ RSA؟

✦ RSA بطيئة جداً وتشفر فقط بيانات أصغر من حجم المفتاح. فنشفر البيانات بـ AES السريع، ونستخدم RSA لتشفير **مفتاح AES الصغير فقط**. هذا هو الأسلوب المعتمد عالمياً (مثل HTTPS/TLS).

? ما OAEP؟ ولماذا مفتاح RSA بطول 2048؟

✦ OAEP أسلوب حشو آمن يضيف عشوائية قبل تشفير RSA لمنع الهجمات. وطول 2048 بت هو الحد الموصى به حاليًا (آمن ومتوازن في الأداء)؛ 1024 أصبح ضعيفًا.

? لماذا أضف HMAC إلى 3DES تحديدًا؟ وما Encrypt-then-MAC؟

✦ لأن 3DES في وضع CBC لا يوفر توثيقًا، فأضفت HMAC-SHA256 يدويًا لكشف التلاعب. **Encrypt-then-MAC** تعني: نشفر أولًا ثم نحسب التوقيع على النص المشفر — وهو الترتيب الأكثر أمانًا المثبت أكاديميًا.

? ما timingSafeEqual ولماذا استخدمتها؟

✦ دالة تقارن قيمتين في زمن ثابت لا يعتمد على موضع أول اختلاف، لمنع هجمات التوقيت (Timing Attacks) التي تستنتج المعلومات من فروق زمن المقارنة.

? لماذا تُعدّ 3DES ضعيفة اليوم؟

✦ حجم كتلتها صغير (64 بت) وأبطأ بكثير، وقد أوقف اعتمادها رسميًا (NIST). أدرجناها للمقارنة التعليمية فقط، مع تعزيزها بـ HMAC.

? ما دور helmet و express-rate-limit؟

✦ **helmet** يضيف رؤوس HTTP أمنية (مثل سياسة المحتوى CSP) للحماية من هجمات الويب. **rate-limit** يحدّ عدد الطلبات من نفس المصدر لمنع إساءة الاستخدام وهجمات الإغراق.

? أين تُخزّن الملفات المرفوعة؟ هل تُحفظ على الخادم؟

✦ تُعالج في الذاكرة فقط (multer memory storage) ولا تُكتب على القرص إطلاقًا، ما يعزّز الخصوصية. تختفي فور انتهاء الطلب.

? لماذا تُرسل البيانات كـ base64 ولماذا رفعت حد JSON إلى 100mb؟

✦ لأن البيانات الشائبة (الملفات/الصور) لا تُنقل كنص عادي، فنحوّلها إلى base64. وبما أن base64 يكبّر الحجم نحو الثلث، رفعنا الحد إلى 100 ميجابايت لاستيعاب ملفات حتى 50 ميجابايت.

? ما هي حدود المشروع وتحسيناته المستقبلية؟

✦ حاليًا التشفير يجري على الخادم؛ مستقبلاً يمكن إضافة تشفير من جهة العميل، ودعم ملفات أكبر عبر التدفق (streaming)، وإضافة توقيع رقمي وإدارة مفاتيح أكثر تقدّمًا.

? لماذا استخدمت Electron؟

✦ لتغليف نفس الخادم والواجهة في برنامج سطح مكتب واحد (EXE) يعمل بلا متصفح ولا إعداد، فيسهل تشغيله على أي جهاز ويندوز.

? ماذا يحدث لو مُقد المفتاح الخاص في RSA؟

✦ تستحيل استعادة البيانات — وهذا سلوك أمني مقصود. المفتاح الخاص هو الوحيد القادر على فكّ التشفير.

(8) تشغيل المشروع وبناء EXE

```
npm install      // تثبيت المكتبات (مرة واحدة)
npm start        // تشغيل كموقع : http://localhost:3000
npm run electron // تشغيل كنافذة سطح مكتب
npm run dist     // (على ويندوز) في مجلد EXE بناء ملف
```

صنع بحب ✦ CipherForge — دليل المشروع